

バーPOS 技術仕様書

開発者・システム担当者向けの仕様書です。計算ロジック、データ構造、基盤、セキュリティ、既知の制約までを記載します。

1. システム概要

iPad のブラウザ上で動作するバー向け POS（注文・会計・売上管理）です。売上データは Firebase Firestore にリアルタイム保存され、オーナーは別端末（PC等）で同じ URL を開くだけで最新の売上を確認できます。

- 注文入力（席ごと、品目単位で担当キャストを設定）
- 会計（現金 / カード / QR、お釣り計算、決済手数料・実入金額の自動計算）
- 売上管理（期間集計、支払方法別、キャストバック集計、CSV出力）
- メニュー / キャスト / 手数料 / バック率をオーナーが画面から管理
- スタッフ / オーナーの権限分離（ログイン）

2. 技術スタック・基盤

項目	内容
フロントエンド	React 18 + TypeScript + Vite 5
状態管理	Zustand 4
認証	Firebase Authentication（メール/パスワード）
データベース	Firebase Firestore
ホスティング	Firebase Hosting
対象端末	iPad Safari（PC ブラウザでも動作）
アイコン	Tabler Icons（CDN 読み込み）

- Firebase プロジェクト ID: `bar-pos-b1993`
- 公開 URL（デプロイ後）: `https://bar-pos-b1993.web.app`
- すべてクライアントサイド SPA。サーバーサイドのコードは無し（バックエンドは Firebase の各サービス）。

構成イメージ

```
[iPad Safari] ┌
               │
               └─> Firebase Hosting（静的SPA配信）
[PC ブラウザ] ┌
               │
               └─> Firebase Authentication（ログイン／ロール判定）
                   └─> Firestore（メニュー/キャスト/設定/取引をリアルタイム同期）
```

3. ディレクトリ構成

```
src/
├── components/
│   ├── LoginScreen.tsx      # ログイン画面
│   ├── OrderScreen.tsx      # 注文入力（席・メニュー・伝票）
│   ├── CheckoutScreen.tsx   # 会計
│   ├── SalesScreen.tsx      # 売上管理（オーナー専用）
│   ├── MenuManageScreen.tsx # メニュー管理（オーナー専用）
│   └── CastManageScreen.tsx # キャスト管理（オーナー専用）
├── hooks/
│   └── useSalesSummary.ts    # 売上・バック集計ロジック
├── lib/
│   ├── firebase.ts          # Firebase 初期化（Firestore + Auth）・コレクション定数
│   ├── authConfig.ts        # ログインID⇄メール変換・ロール判定
│   ├── tax.ts               # 税・手数料の計算ユーティリティ
│   ├── csv.ts               # CSV 生成・ダウンロード
│   └── defaultMenus.ts      # デフォルトメニュー / フリー入力プリセット
├── store/
│   └── posStore.ts           # Zustand グローバル状態・全アクション
├── types/index.ts            # 型定義
├── styles/global.css         # スタイル
├── App.tsx                   # ルート（認証ゲート・画面切替・購読）
├── main.tsx                  # エントリーポイント
└── firestore.rules            # Firestore セキュリティルール
```

4. データモデル（Firestore）

コレクション名は `src/lib/firebase.ts` の `COLLECTIONS` で定義。

`transactions`（会計確定済みの取引）

会計確定時に1件追加（追記のみ。更新・削除はアプリからは行わない）。

フィールド	型	説明
seatName	string	席名（未入力時は「席 A」等）
solo	boolean	一人客フラグ
items	OrderItem[]	注文品目の配列（各品目に担当キャスト名を保持）
subtotal	number	税抜合計
tax	number	消費税額
total	number	税込合計
payMethod	'cash' 'card' 'qr'	支払方法
feeRate	number	決済手数料率（%）
feeAmount	number	決済手数料額
netAmount	number	実入金額（total - feeAmount）
primaryCast	string	主担当キャスト（売上最大の担当。CSV表示用）
completedAt	number	会計時刻（Unix ミリ秒）
_createdAt	Timestamp	サーバータイムスタンプ（書き込み時刻）

OrderItem（items 配列の要素）：

フィールド	型	説明
id	string	一意ID
name	string	品名
priceExTax	number	税抜単価
qty	number	数量
cast	string	担当キャスト名（空文字=未設定）
isToday	boolean	本日限定メニュー
isFree	boolean	フリー入力

menus（メニュー）

{ name, priceExTax, category, isToday, sortOrder }。sortOrder 昇順で表示。本日限定は isToday: true ・ category: '本日限定'。

casts（キャスト）

{ name, sortOrder }。sortOrder 昇順で表示。注文画面の担当選択肢になる。

settings（設定。ドキュメントIDで区別）

- settings/fees ... { card: number, qr: number }（各%）
- settings/back ... { rate: number }（0.30 = 30%）

重要: 席 (seat) と未会計の注文 (orders) は **Firestore に保存していません** (端末メモリ上のみ)。詳細は「10. 既知の制約」を参照。

5. 認証・権限

ログイン方式

Firebase Authentication の「メール/パスワード」を使用。現場で打ちやすいよう、ログイン画面では **ユーザーID** だけを入力し、内部でメールアドレスへ変換する (`src/lib/authConfig.ts`)。

- 変換: **ID** → **ID@<VITE_AUTH_ID_DOMAIN>** (既定ドメイン `bar-pos.local`)
- 例: ログインID `owner` → `owner@bar-pos.local`

ロール判定

ログイン中のメールアドレスで自動判定 (DB にロールを持たせない)。

```
role = (email === `${VITE_OWNER_ID}@${VITE_AUTH_ID_DOMAIN}`) ? 'owner' : 'staff'
```

- `owner@bar-pos.local` → **オーナー** (全画面)
- それ以外 → **スタッフ** (注文入力・会計のみ)

アプリ側 (画面の出し分け) と Firestore ルール (DB アクセス制御) の **両方** が同じメールで判定するため、UI を隠すだけでなく DB レベルでも保護される。

Firestore セキュリティルール (`firestore.rules`)

コレクション	read	write
<code>menus</code>	ログイン済み	オーナーのみ
<code>casts</code>	ログイン済み	オーナーのみ
<code>settings</code>	ログイン済み	オーナーのみ
<code>transactions</code>	オーナーのみ	create はログイン済み (=スタッフも会計確定可)、update/delete はオーナーのみ

→ スタッフのアカウントでは (UI だけでなく) DB から直接でも過去売上を読めない。

ルール内のオーナー判定は `owner@bar-pos.local` をハードコードしている。 `VITE_OWNER_ID` / `VITE_AUTH_ID_DOMAIN` を変えたら、ルールの該当箇所も `<VITE_OWNER_ID>@<VITE_AUTH_ID_DOMAIN>` に合わせることを。

6. 計算ロジック

実装は `src/lib/tax.ts` (税・手数料)、`src/store/posStore.ts` (会計確定)、`src/hooks/useSalesSummary.ts` (集計)。

6.1 消費税（税率 10%）

明細は **税抜** で表示し、消費税は **小計（税抜合計）に対して1回だけ** 計算する（端数は切り捨て）。これにより「明細の合計＝小計＝合計欄」が常に一致する。

```
小計（税抜） subtotal = Σ(priceExTax × qty)
消費税      tax      = floor(subtotal × 0.10)
合計（税込） total    = subtotal + tax
```

- メニューボタンの単価表示のみ税込（ `toTaxInc(p) = floor(p × 1.10)` ）＝お客様向けの総額表示。
- 会計レシート/伝票の明細は税抜、消費税は最後に1回。

例: 税抜 545 円のドリンク × 3

```
subtotal = 545 × 3 = 1,635
tax       = floor(1,635 × 0.1) = 163
total     = 1,798
```

6.2 決済手数料・実入金額

```
feeRate  = (cash: 0, card: settings.fees.card, qr: settings.fees.qr) // %
feeAmount = floor(total × feeRate / 100)
netAmount = total - feeAmount
```

既定値: カード 3.25%、QR 1.98%（オーナーが画面から変更可）。現金は手数料 0。

例: total 10,000 ・ カード 3.25%

```
feeAmount = floor(10,000 × 0.0325) = 325
netAmount = 9,675
```

6.3 お釣り（現金）

```
change = cashReceived - total // cashReceived ≥ total のときのみ確定可能
```

預かり金額は `total` 以上のプリセット（1,000/2,000/5,000/10,000+ぴったり額）から選択。

6.4 キャストバック（按分・税抜基準）

会計時に各品目の担当キャスト（ `item.cast` ）へ **税抜売上** を割り当て、キャストごとに合算してバック率を掛ける（端数は四捨五入）。

```
(キャスト別) 税抜売上 castSales[cast] = Σ(priceExTax × qty) // そのキャスト担当の品目のみ
バック額      backAmount[cast]      = round(castSales[cast] × backRate)
```

- バック率 `backRate` は `settings/back`（画面設定）→ `VITE_BACK_RATE` → 既定 0.30 の優先順位。
- 担当未設定（ `cast === ''` ）の品目は「未設定」としてまとめ、**バック対象外（0円）**。タグ漏れに気づけるよう集計には表示。

- ・ 同卓で複数キャストが担当している場合も、品目ごとに正しく按分される（主担当1人に寄せない）。
- ・ 担当件数（txCount）は「そのキャストが関与した取引数」。

例: 同一会計で さくら担当品目 税抜3,000円 / あおい担当品目 税抜2,000円・バック率30%

```
さくら back = round(3,000 × 0.30) = 900
あおい back = round(2,000 × 0.30) = 600
```

6.5 売上サマリー（ useSalesSummary ）

期間内の transactions から集計:

```
totalSales      = Σ total          // 税込
totalFee        = Σ feeAmount
totalNet        = totalSales - totalFee
txCount         = 取引件数
avgPerCustomer  = round(totalSales / txCount) // 1取引あたり（人数ベースではない）
soloCount       = solo の件数
byMethod[m]     = { count, sales(=Σtotal), fee, net } を支払方法別に集計
castSummaries   = 6.4 の按分結果（税抜売上ベース）
```

注: キャスト別「売上(税抜)」の合計は、上部「売上合計(税込)」とは基準が異なるため一致しない（仕様。見出しで区別している）。

6.6 期間の範囲（ SalesScreen.periodRange ）

期間	from	to
今日	当日 00:00:00	当日 23:59:59.999
今週	6日前の 00:00:00（直近7日のローリング）	当日 23:59:59.999
今月	当月1日 00:00:00	当日 23:59:59.999

「今週」はカレンダー一週ではなく直近7日である点に注意。

7. リアルタイム同期

onSnapshot で購読:

- ・ menus ・ casts : ログイン後に全ユーザーが購読。
- ・ transactions : 売上画面（オーナー）で、選択期間を where(completedAt >= from && <= to) で絞り込み購読（全件取得はしない）。

会計確定は addDoc（追記）。確定後、その席の注文（メモリ）をクリア。

8. CSV 出力（ src/lib/csv.ts ）

- ・ BOM 付き UTF-8（Excel で文字化けせず開ける）。

- ・ 売上CSV: 日時/席名/一人客/支払方法/税抜売上/消費税/税込売上/手数料率/手数料額/実入金額/主担当キャスト。
- ・ バックCSV: キャスト/担当件数/売上合計(税抜)/バック額。

9. 環境変数 (`.env`)

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
VITE_AUTH_ID_DOMAIN=bar-pos.local # ログインID→メール変換ドメイン
VITE_OWNER_ID=owner # オーナー扱いにするログインID
VITE_BACK_RATE=0.30 # バック率の初期値 (画面保存値が優先)
```

`.env` は `.gitignore` 済み (Firebase キーを含むためリポジトリに含めない)。

10. 既知の制約・注意点

実運用前に把握しておくべき項目 (現時点で未対応)。

1. **席・未会計の注文は端末メモリのみ** (Firestore 非保存)。
 - ・ ページのリロード/Safari のタブ破棄/端末再起動で **開いている伝票が消える**。
 - ・ 複数端末で同じ未会計伝票を共有できない。
 - ・ → 営業中は会計が済むまでリロードしない運用が必要。
2. **取引の取消・返金機能が無い**。打ち間違いはオーナーが Firestore を直接修正する。
3. **オフライン耐性が限定的**。会計確定時にネットワーク失敗してもアプリ上に明確なエラー表示が出ない (Firestore SDK のキュー任せ)。
4. **CSV インジェクション対策なし**。席名等に `=`, `+`, `-`, `@` で始まる文字列を入れると、Excel で数式として解釈され得る。
5. **「今週」は直近7日** (カレンダー週ではない)。
6. **客単価は1取引あたり** (来店人数ベースではない)。
7. **キャストのリネーム/削除は過去データに遡及しない**。取引内には会計時点のキャスト名 (文字列) が保存されるため、改名後も過去記録は旧名のまま。

11. セットアップ・デプロイ

詳細は [README.md](#) を参照。要点のみ:

```
npm install
cp .env.example .env # Firebase 設定値とログイン設定を記入
npm run dev           # http://localhost:5173

# デプロイ
npm run build
firebase deploy       # Hosting (firebase.json 設定済み)
```

- Firestore ルールは `firestore.rules` の内容を Firebase Console → Firestore → ルール に貼って「公開」（または CLI で反映する場合は `firebase.json` に firestore 設定を追加）。
- Authentication で「メール/パスワード」を有効化し、`owner@bar-pos.local` / `staff@bar-pos.local` を作成。
- メニュー・キャストの初期データは、オーナーでログインして各管理画面の「投入」ボタンから登録。

12. 今後の改善候補

- 席・注文の Firestore 永続化（リロード耐性・複数端末共有）
- 取引の取消・返金フロー
- オフライン永続化（`persistentLocalCache`）と会計失敗時のエラー表示
- CSV インジェクション対策（先頭エスケープ）
- 席バック／商品バックなど複数バック率への拡張